

Mini Guida Pratica: Il Polimorfismo in C++

Istruzioni, regole d'oro e promemoria per gli esercizi della concessionaria e della fabbrica

Il **Polimorfismo** è il terzo pilastro della programmazione a oggetti (dopo l'incapsulamento e l'ereditarietà). Permette a classi figlie diverse di rispondere allo stesso identico comando in modi differenti e personalizzati.

1. I Due Comandi Chiave: `virtual` e `override`

Per attivare il polimorfismo, dobbiamo preparare sia la classe padre che le classi figlie con due parole chiave fondamentali:

- **`virtual` (nella classe Padre):** Si mette prima del tipo di ritorno del metodo. Dice al compilatore: *"Attenzione, questo metodo non è fisso, può essere riscritto dalle classi figlie!"*.
- **`override` (nelle classi Figlie):** Si mette alla fine della firma del metodo. Non è obbligatorio, ma è una protezione fondamentale: se sbagli a scrivere il nome del metodo, il compilatore ti blocca subito segnalando l'errore.

2. La Regola d'Oro: Puntatori (*) e Memoria Dinamica (`new`)

In C++, il polimorfismo **funziona solo tramite i puntatori** (o i riferimenti). Se assegni un oggetto figlio a una variabile padre normale, subentra lo *Slicing* (il C++ taglia via le parti specifiche della figlia per far entrare l'oggetto nello spazio del padre, distruggendo il polimorfismo).

Come fare correttamente: Si crea un puntatore del tipo del Padre, ma lo si punta a una nuova istanza della classe Figlia:

```
ClassePadre* mio_oggetto = new ClasseFiglia("Nome");
```

3. La Sintassi della Freccia (`->`)

Quando hai in mano un puntatore (una variabile con l'asterisco), non puoi usare il punto (`.`) per chiamare i metodi. Devi usare la freccia (`->`).

- `oggetto.metodo();` → Si usa con gli oggetti normali (allocati sullo stack).
- `puntatore->metodo();` → Si usa con i puntatori (allocati dinamicamente con `new`). Rappresenta il comando: *"vai a quell'indirizzo di memoria ed esegui il metodo dell'oggetto reale"*.

4. Il Distruttore Virtuale: Evitare i Memory Leak

IMPORTANTE: Ogni volta che dichiari anche una sola funzione `virtual` nella classe Padre, devi **sempre** aggiungere il distruttore virtuale nel Padre:

```
virtual ~NomeClassePadre() {}
```

Se non lo metti, quando userai il comando `delete`, il C++ eliminerà solo i pezzi dell'oggetto legati al padre, lasciando i pezzi della figlia bloccati in memoria (causando un *memory leak*).

5. Struttura Modello Completa

Ecco lo schema vuoto perfetto da usare come riferimento visivo per i tuoi esercizi:

```
// 1. CLASSE PADRE
class Padre {
public:
    string nome;

    Padre(string n) { nome = n; }

    // Regola d'oro: Distruttore virtuale sempre presente
    virtual ~Padre() {}

    // Metodo virtuale pronto per essere sovrascritto
    virtual void esegui_azione() {
        cout << nome << " fa un'azione generica." << endl;
    }
};

// 2. CLASSE FIGLIA
class Figlia : public Padre {
public:
    // Il costruttore chiama quello del padre
    Figlia(string n) : Padre(n) {}

    // Sovrascrittura sicura con 'override'
    void esegui_azione() override {
        cout << nome << " fa l'azione specifica della figlia!" << endl;
    }
};

// 3. UTILIZZO NEL MAIN
int main() {
    // Creazione del puntatore polimorfico
    Padre* ptr = new Figlia("Esempio");

    // Chiamata con la freccia ->
    ptr->esegui_azione(); // Esegue il metodo della FIGLIA

    // Liberazione obbligatoria della memoria
    delete ptr;

    return 0;
}
```